

1、项目描述

请先整体描述项目实现的功能
再按模块分别描述

2、简述一下C++中的多态

在现实生活中，多态是同一个事物在不同场景下的多种形态。

在面向对象中，多态是指通过基类的指针或者引用，在运行时动态调用实际绑定对象函数的行为，与之相对应的编译时绑定函数称为静态绑定。所以多态分为静态多态和动态多态。

1. 静态多态 静态多态是编译器在编译期间完成的，编译器会根据实参类型来选择调用合适的函数，如果有合适的函数就调用，没有的话就会发出警告或者报错。静态多态有函数重载、运算符重载、泛型编程等。
2. 动态多态 动态多态是在程序运行时根据基类的引用（指针）指向的对象来确定自己具体该调用哪一个类的虚函数。当父类指针（引用）指向 父类对象时，就调用父类中定义的虚函数；即当父类指针（引用）指向 子类对象时，就调用子类中定义的虚函数。

3、简述一下深拷贝与浅拷贝

1. 浅拷贝 浅拷贝又称为值拷贝，将源对象的值拷贝到目标对象中，如果对象中有某个成员是指针类型数据，并且是在堆区创建，则使用浅拷贝仅仅拷贝的是这个指针变量的值，也就是在目标对象中该指针类型数据和源对象中的该成员指向的是同一块堆空间。这样会带来一个问题，就是在析构函数中释放该堆区数据，会被释放多次。默认的拷贝构造函数和默认的赋值运算符重载函数都是浅拷贝。
2. 深拷贝 深拷贝在拷贝的时候先开辟出和源对象大小一样的空间，然后将源对象里的内容拷贝到目标对象中去，这样指针成员就指向了不同的内存位置。并且里面的内容是一样的，这样不但达到了拷贝的目的，还不会出现问题，两个对象先后去调用析构函数，分别释放自己指针成员所指向的内存。深拷贝情况下，不会出现重复释放同一块内存的错误。

4、STL容器用过哪些？查找的时间复杂度是多少？

STL 中常用的容器有 vector、deque、list、map、set、multimap、multiset、unordered_map、unordered_set 等。

容器底层实现方式及时间复杂度分别如下：

1. vector 采用一维数组实现，元素在内存连续存放，不同操作的时间复杂度为：插入: $O(N)$ 查看: $O(1)$ 删除: $O(N)$
2. deque 采用双向队列实现，元素在内存连续存放，不同操作的时间复杂度为：插入: $O(N)$ 查看: $O(1)$ 删除: $O(N)$
3. list 采用双向链表实现，元素存放在堆中，不同操作的时间复杂度为：插入: $O(1)$ 查看: $O(N)$ 删除: $O(1)$
4. map、set、multimap、multiset 上述四种容器采用红黑树实现，红黑树是平衡二叉树的一种。不同操作的时间复杂度近似为：插入: $O(\log N)$ 查看: $O(\log N)$ 删除: $O(\log N)$
5. unordered_map、unordered_set、unordered_multimap、unordered_multiset 上述四种容器采用哈希表实现，不同操作的时间复杂度为：插入: $O(1)$ ，最坏情况 $O(N)$ 查看: $O(1)$ ，最坏情况 $O(N)$ 删除: $O(1)$ ，最坏情况 $O(N)$
注意：容器的时间复杂度取决于其底层实现方式。

5、说说STL中容器的类型，每种分别有哪些容器？

1. 序列式容器 array、vector、deque、list、forward_list
2. 关联式容器 map、multimap、set、multiset
3. 无序关联式容器 unordered_map、unordered_multimap、unordered_set、unordered_multiset
4. 容器适配器 stack、queue、priority_queue

6、简述一下C++的重载与重写

- 重载

- a. 重载是指不同的函数使用相同的函数名，但是函数的参数个数或类型不同（参数列表不同）。调用的时候根据函数的参数来区别不同的函数，函数重载跟返回值无关。
- b. 重载的规则 - 作用域相同 - 函数名相同 - 必须具有不同的参数列表
- c. 重载用来实现静态多态（函数名相同，功能不一样）。

- 重写

- a. 重写（也叫覆盖）是指在派生类中重新对基类中的虚函数重新实现。即函数名和参数都一样，只是函数的实现体不一样。
- b. 重写的规则： - 方法声明必须完全与父类中被重写的方法相同 - 子类重写的方法可以加virtual，也可以不加
- c. 重写用来实现动态多态（根据调用方法的对象的类型来执行不同的函数）。

7、为什么将析构函数设置成虚函数？

1. 概念--虚析构函数，是将基类的析构函数声明为 virtual ,如：

```
class Base
{
    public: Base() { }
    // 虚析构函数
    virtual ~Base() { }
}
```

2. 作用--虚析构函数的主要作用是为了防止遗漏资源的释放，防止内存泄露。

如果基类中的析构函数没有声明为虚函数，基类指针指向派生类对象时，则当基类指针释放时不会调用派生类对象的析构函数，而是调用基类的析构函数，如果派生类析构函数中做了某些释放资源的操作，则这时就会造成内存泄露。

8、说说map和set的区别

map和set都是C++的关联容器，其底层实现都是红黑树（RB-Tree）

map和set区别在于：

1. map中的元素是key-value（关键字—值）对：关键字起到索引的作用，值则表示与索引相关联的数据；Set与之相对就是关键字的简单集合，set中每个元素只包含一个关键字。
2. set的迭代器是const的，不允许修改元素的值；map允许修改value，但不允许修改key。

其原因是因为map和set是根据关键字排序来保证其有序性的，如果允许修改key的话，那么首先需要删除该键，然后调节平衡，再插入修改后的键值，调节平衡，如此一来，严重破坏了map和set的结构，导致iterator失效，不知道应该指向改变前的位置，还是指向改变后的位置。所以STL中将set的迭代器设置成const，不允许修改迭代器的值；而map的迭代器则不允许修改key值，允许修改value值。

3. map支持下标操作，set不支持下标操作。

map可以用key做下标，map的下标运算符[]将关键码作为下标去执行查找，如果关键码不存在，则插入一个具有该关键码和mapped_type类型默认值的元素至map中，因此下标运算符[]在map应用中需要慎用，const_map不能用，只希望确定某一个关键值是否存在而不希望插入元素时也不应该使用，mapped_type类型没有默认值也不应该使用。如果find能解决需要，尽可能用find。

9、说说vector的扩容机制，扩容后，它的内存地址会变化吗？

当 vector 的大小和容量相等（size==capacity）也就是满载时，如果再向其添加元素，那么 vector 就需要扩容。

vector 容器扩容的过程需要经历以下 3 步：

1. 完全弃用现有的内存空间，重新申请更大的内存空间；
2. 将旧内存空间中的数据，按原有顺序移动到新的内存空间中；
3. 最后将旧的内存空间释放。

因为 vector 扩容需要申请新的空间，所以扩容以后它的内存地址会发生改变。

vector 扩容是非常耗时的，为了降低再次分配内存空间时的成本，每次扩容时 vector 都会申请比用户需求量更多的内存空间（这也就是 vector 容量的由来，即 capacity>=size），以便后期使用。

10、说说C++中智能指针和指针的区别是什么？

- 智能指针

如果在程序中使用 new 从堆（自由存储区）分配内存，等到不需要时，应使用 delete 将其释放。

C++ 引入了智能指针 auto_ptr，以帮助自动完成这个过程。随后的编程体验（尤其是使用STL）表明，需要有更精致的机制。基于程序员的编程体验和 BOOST 库提供的解决方案，C++11 摒弃了 auto_ptr，并新增了三种智能指针：unique_ptr、shared_ptr 和 weak_ptr。所有新增的智能指针都能与 STL 容器和移动语义协同工作。

- 指针

C 语言规定所有变量在使用前必须先定义，指定其类型，并按此分配内存单元。指针变量不同于整型变量和其他类型的变量，它是专门用来存放地址的，所以必须将它定义为“指针类型”。

- 智能指针和普通指针的区别

智能指针和普通指针的区别在于智能指针实际上是对普通指针加了一层封装机制，区别是它负责自动释放所指的对象，这样的一层封装机制的目的是为了使得智能指针可以方便的管理一个对象的生命期。指针是一种数据类型，用于保存内存地址，而智能指针是类模板。

11、请你说说迭代器失效原因，有哪些情况

STL 中某些容器调用了某些成员方法后会导致迭代器失效。例如 vector 容器，如果调用 reserve() 来增加容器容量，之前创建好的任何迭代器（例如开始迭代器和结束迭代器）都可能会失效，这是因为，为了增加容器的容量，vector 容器的元素可能已经被复制或移到了新的内存地址。

- 序列式容器迭代器失效

对于序列式容器，例如 vector、deque，由于序列式容器是组合式容器，当当前元素的迭代器被删除后，其后的所有元素的迭代器都会失效，这是因为 vector、deque 都是连续存储的一段空间，所以当对其进行 erase 操作时，其后的每一个元素都会向前移一个位置。

- 关联式容器迭代器失效

对于关联容器，例如如 map、set，删除当前的迭代器，仅仅会使当前的迭代器失效。这是因为 map 之类的容器，使用了红黑树来实现，插入、删除一个节点不会对其他点造成影响。

12、请你说说虚函数可以是内联函数吗

1. 虚函数可以是内联函数，内联是可以修饰虚函数的，但是当虚函数表现多态性的时候不能内联。
2. 内联是在编译期建议编译器内联，而虚函数的多态性在运行期，编译器无法知道运行期调用哪个代码，因此虚函数表现为多态性时不可以内联。
3. inline virtual 唯一可以内联的时候是：编译器知道所调用的对象是哪个类，这只有在编译器具有实际对象而不是对象的指针或引用时才会发生。

13、简述一下什么是面向对象

面向对象思想是基于面向过程思想的，要说面向对象思想，先说说面向过程思想。

1. 面向过程思想

完成一个需求的步骤：首先是搞清楚要做什么，然后再分析怎么做，最后再通过代码体现。一步一步去实现，而具体的每一步都需要我们去实现和操作。这些步骤相互调用和协作，从而完成需求。在上面的每一个具体步骤中我们都是参与者，并且需要面对具体的每一个步骤和过程，这就是面向过程最直接的体现。面向过程编程，其实就是面向着具体的每一个步骤和过程，把每一个步骤和过程完成，然后由这些功能函数相互调用，完成需求。

2. 面向对象思想

面向对象的思想是尽可能模拟人类的思维方式，使得软件的开发方法与过程尽可能接近人类认识世界、解决现实问题的方法和过程，把客观世界中的实体抽象为问题域中的对象。面向对象以对象为核心，该思想认为程序由一系列对象组成。面向对象思想的特点：- 是一种更符合人类思维习惯的思想 - 可以将复杂的问题简单化 - 将我们从执行者变成了指挥者

3. 面向对象的三大特征：封装、继承、多态

封装：将事物属性和行为封装到一起，也就是 C++ 中的类，便于管理，提高代码的复用性。事物的属性和行为分别对应类中的成员变量和成员方法。

继承：继承使类与类之间产生关系，能够提高代码的复用性以及可维护性。

多态：多态意味着调用成员函数时，会根据调用方法的对象的类型来执行不同的函数。

4. 加分回答

面向过程和面向对象解决问题举例：以洗衣服为例。 1. 面向过程：接水到盆中 —— 放入衣服 —— 加入洗衣粉 —— 浸泡 —— 搓衣服 —— 过水 —— 拧干 —— 晾干 2. 面向对象：将衣服放入洗衣机 —— 加入洗衣粉 —— 开启 —— 晾干 通过例子可以发现面向对象的方式解决问题更加简单一些，但是面向对象还是基于面向过程的。